

本讲内容

√ 关联规则

u Write-based、DFS算法

– VIPER

- | Implement DFS with bitmap compression of data in vertical format

– FP-Growth算法

- | Implement DFS using a tree structure in horizontal format

– FP-Growth based sub-graph mining

Comparison of Data Layouts

TID	ItemID
	1 2 3 4 5 ---
1	1 0 1 0 0 ---
2	0 1 0 0 0 ---
3	0 1 1 0 0 ---
4	1 0 1 0 1 ---

(a) HIV

TID	ItemIDs
1	1 3 7 9
2	2 8 15
3	2 3 7 8 11
4	1 3 5 10

(b) HIL

TID	ItemIDs
	1 2 3 4 --
1	1 0 1 0
2	0 1 0 0
3	0 1 1 0
4	1 0 1 0

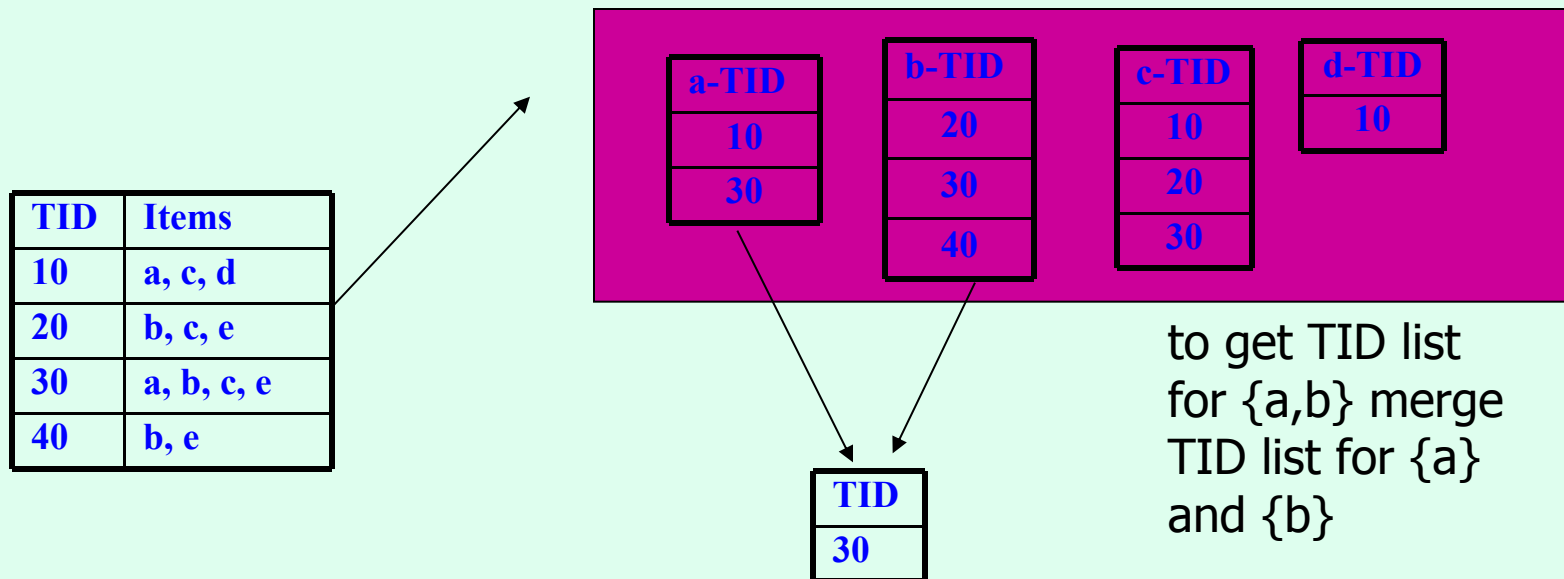
(c) VTV

TIDs	ItemIDs				
	1	2	3	4	--
	1	2	1		
	4	3	3		
			4		

(d) VTL

Vertical format for write-based FPM

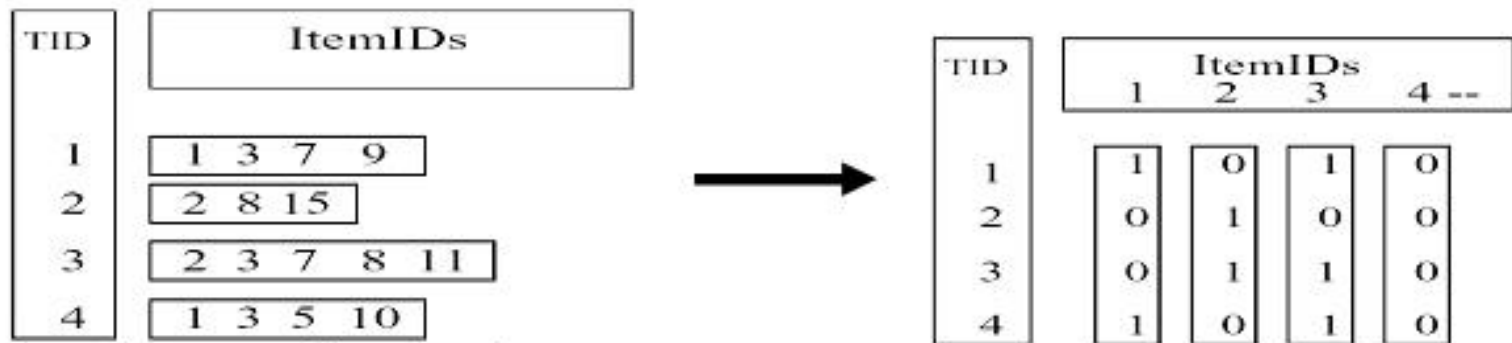
v Vertical format



- Problem: TID lists can be large since each TID will take up $\log_2 N$ bits, N =number of transactions.

Vertical format for write-based FPM

- Solution: Use bitmap encoding



- Golomb encoding scheme

- Divide runs of 1's and 0's into group of fixed size W_0, W_1
- **Weight bits**: Each FULL group represented with a single bit set to "1"
- **Count field**: Last partial group represent using $\log_2 W_i$ bits
- Use "0" as field separator to separate weight bits and count field

本讲内容

√ 关联规则

u Write-based、DFS算法

– VIPER

- | Implement DFS with bitmap compression of data in vertical format

– FP-Growth算法

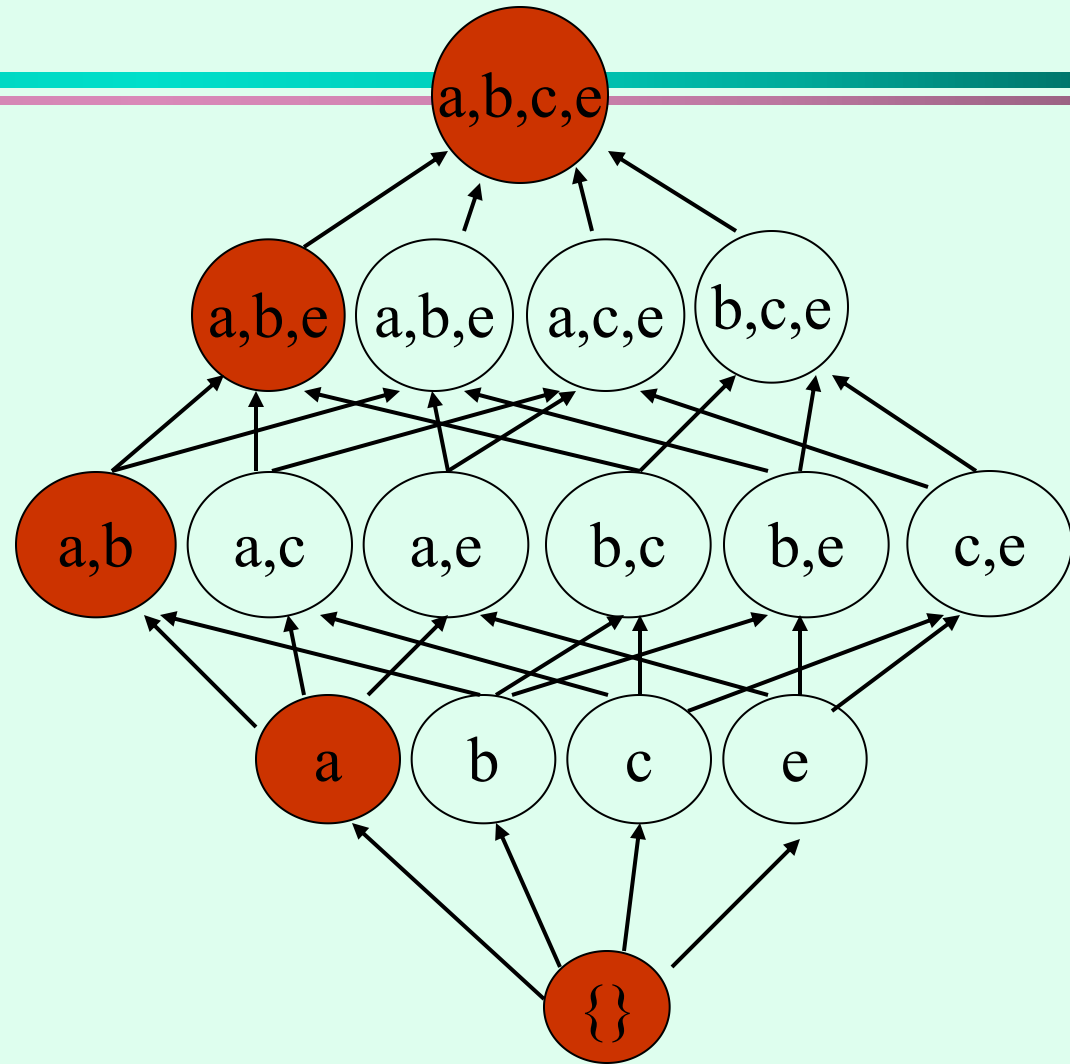
- | Implement DFS using a tree structure in horizontal format

– FP-Growth based sub-graph mining

FP-Growth算法

- **Depth-first search**

- Algorithm with **write** or **point**



FP-Growth算法

Note that we can know the number of occurrence of "ab" by counting the number of "b" in the {a}-projected database !

TID	Items
30	c, e

all rows that contain "ab".

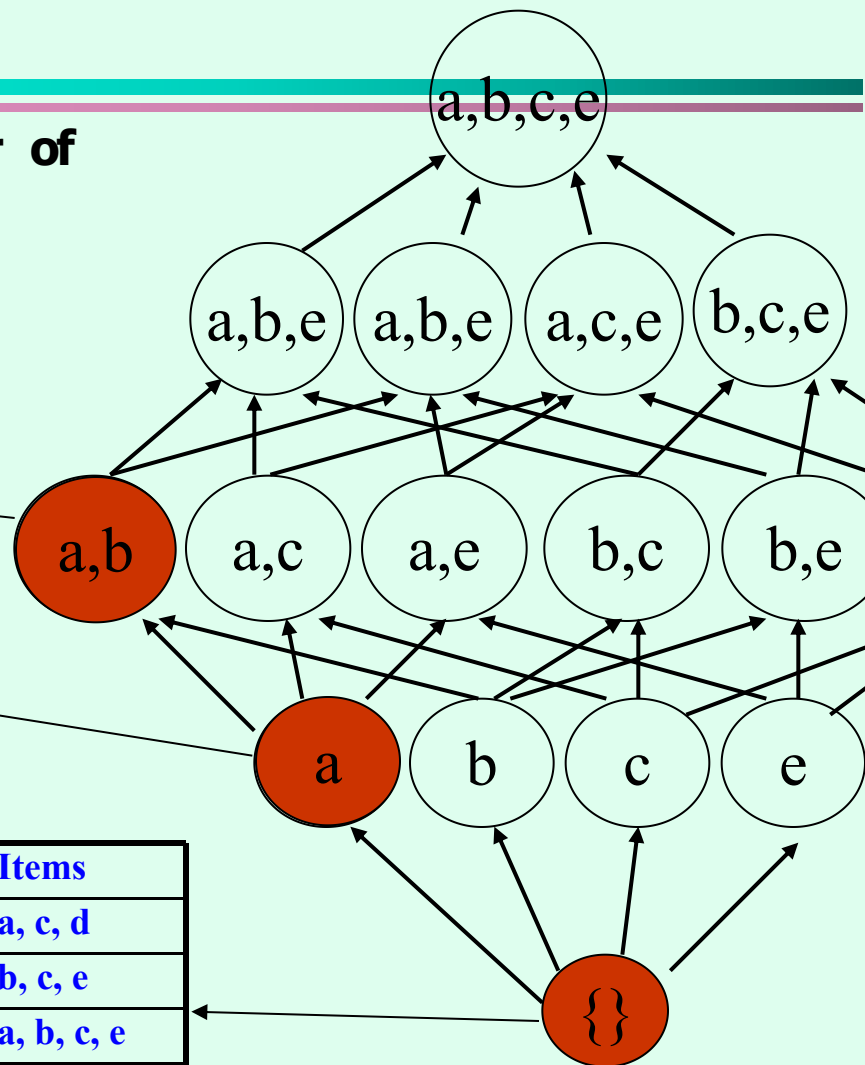
{ab}-projected database

TID	Items
10	c, d
30	b, c, e

all rows that contain "a".

{a}-projected database

TID	Items
10	a, c, d
20	b, c, e
30	a, b, c, e
40	b, e



FP-Growth算法

1. 第一次扫描数据库:

类似于Apriori算法，找出频繁的1-itemset和他们的计数值，将频繁项目按频度降序排列

2. 第二次扫描数据库:

构造fp-tree (频度从大到小)

挖掘该树 (频度从小到大)

注意：数据库大时，fp-tree可能在内存中装不下，需要采取partition方法。

Construct FP-tree from a Transaction Database

<i>TID</i>	<i>Items bought</i>	<i>(ordered) frequent items</i>
100	{ <i>f, a, c, d, g, i, m, p</i> }	{ <i>f, c, a, m, p</i> }
200	{ <i>a, b, c, f, l, m, o</i> }	{ <i>f, c, a, b, m</i> }
300	{ <i>b, f, h, j, o, w</i> }	{ <i>f, b</i> }
400	{ <i>b, c, k, s, p</i> }	{ <i>c, b, p</i> }
500	{ <i>a, f, c, e, l, p, m, n</i> }	{ <i>f, c, a, m, p</i> }

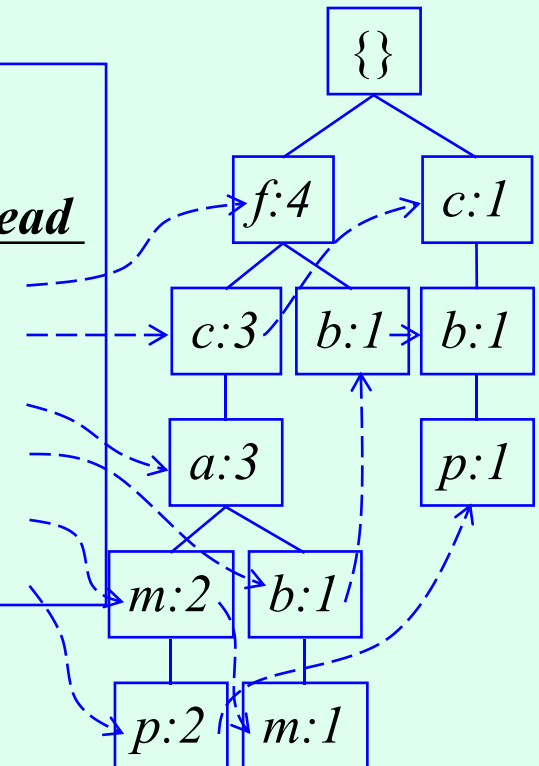
$min_support = 3$

1. 扫描数据库一遍，找到所有的频繁项目
2. 将频繁项目按照降序排列，生成一个f-list
3. 再扫描数据库一遍，构造FP-Tree

Header Table

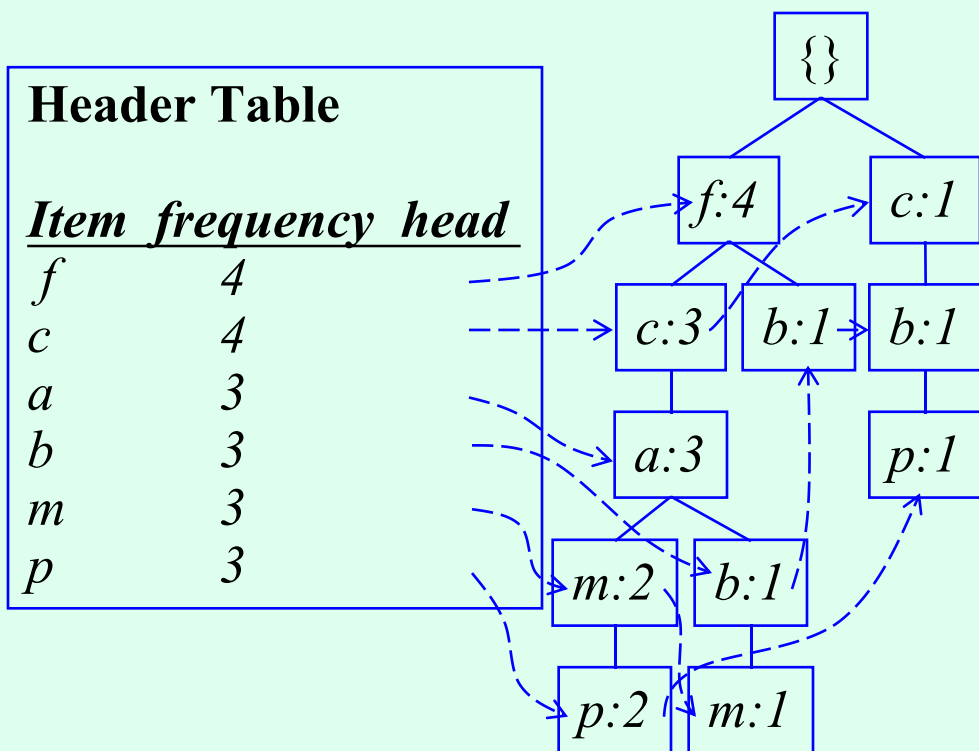
<i>Item</i>	<i>frequency</i>	<i>head</i>
<i>f</i>	4	
<i>c</i>	4	
<i>a</i>	3	
<i>b</i>	3	
<i>m</i>	3	
<i>p</i>	3	

F-list=f-c-a-b-m-p



条件模式基

- 对所有的频繁项目，找到每一个项目在FP-Tree的所有前缀，构成每个项目的条件模式基（类似于条件或投影数据库，但去掉了部分不频繁的项目，如g, i等）



Conditional pattern bases

<i>item</i>	<i>cond. pattern base</i>
-------------	---------------------------

f

c

a

b

m

p

f:3

fc:3

fca:1, f:1, c:1

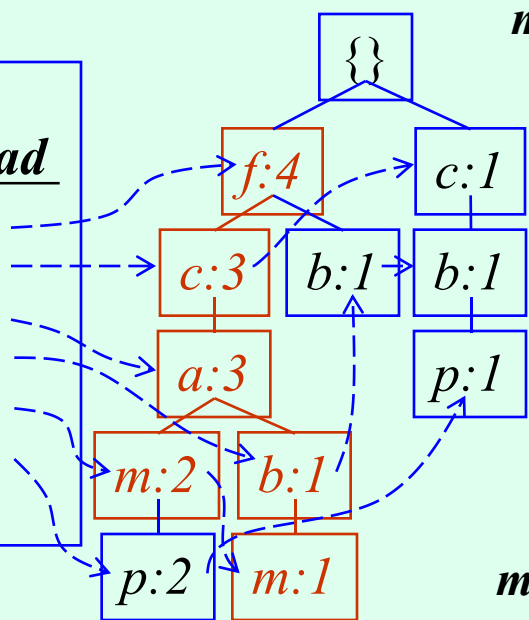
fca:2, fcab:1

fcam:2, cb:1

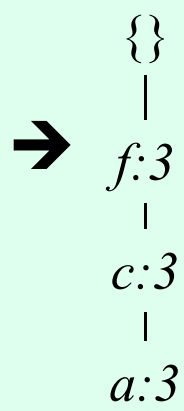
递归构造条件模式基

用每一个项目的条件模式基作为输入，递归FP-Growth算法

Header Table		
<i>Item</i>	<i>frequency</i>	<i>head</i>
<i>f</i>	4	
<i>c</i>	4	
<i>a</i>	3	
<i>b</i>	3	
<i>m</i>	3	
<i>p</i>	3	



m-conditional pattern base:
fca:2, fcab:1



m-conditional FP-tree

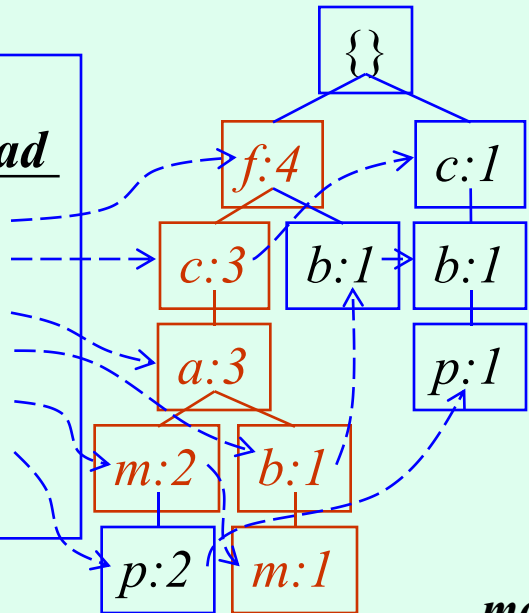
Conditional pattern bases

<i>item</i>	<i>cond. pattern base</i>
<i>mf</i>	
<i>mc</i>	<i>f:3</i>
<i>ma</i>	<i>fc:3</i>

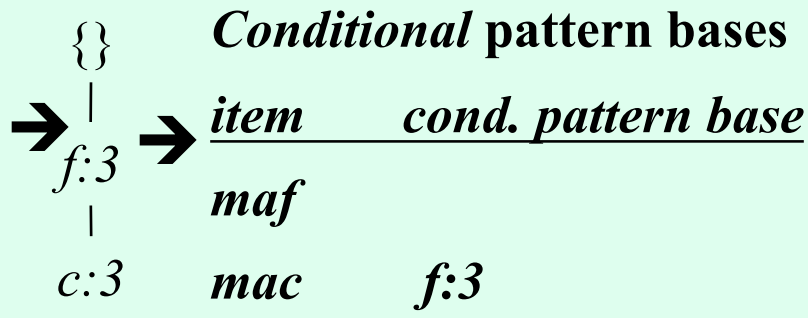
递归构造条件模式基

用每一个项目的条件模式基作为输入，递归FP-Growth算法

Header Table		
<i>Item</i>	<i>frequency</i>	<i>head</i>
<i>f</i>	4	
<i>c</i>	4	
<i>a</i>	3	
<i>b</i>	3	
<i>m</i>	3	
<i>p</i>	3	



ma-conditional pattern base:
fc:3

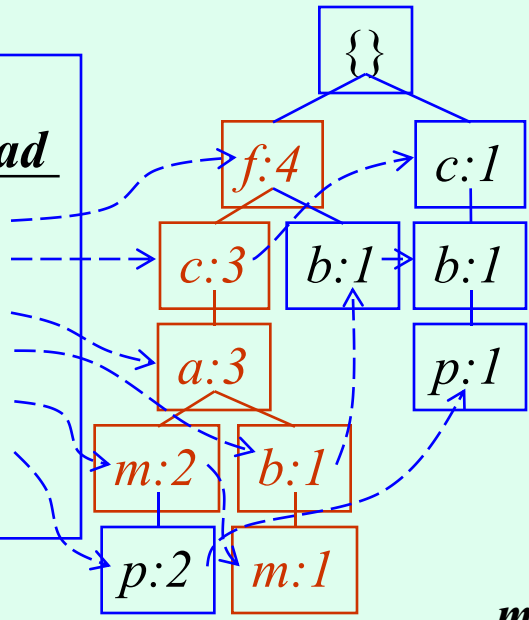


ma-conditional FP-tree

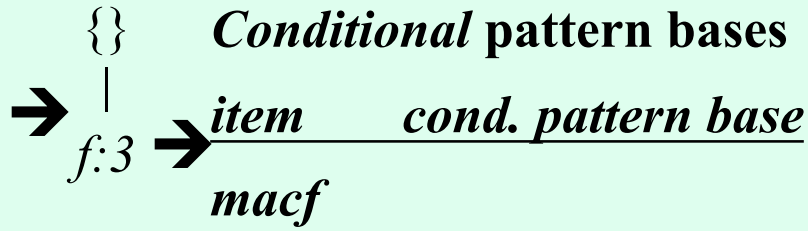
递归构造条件模式基

用每一个项目的条件模式基作为输入，递归FP-Growth算法

Header Table		
<i>Item</i>	<i>frequency</i>	<i>head</i>
<i>f</i>	4	
<i>c</i>	4	
<i>a</i>	3	
<i>b</i>	3	
<i>m</i>	3	
<i>p</i>	3	



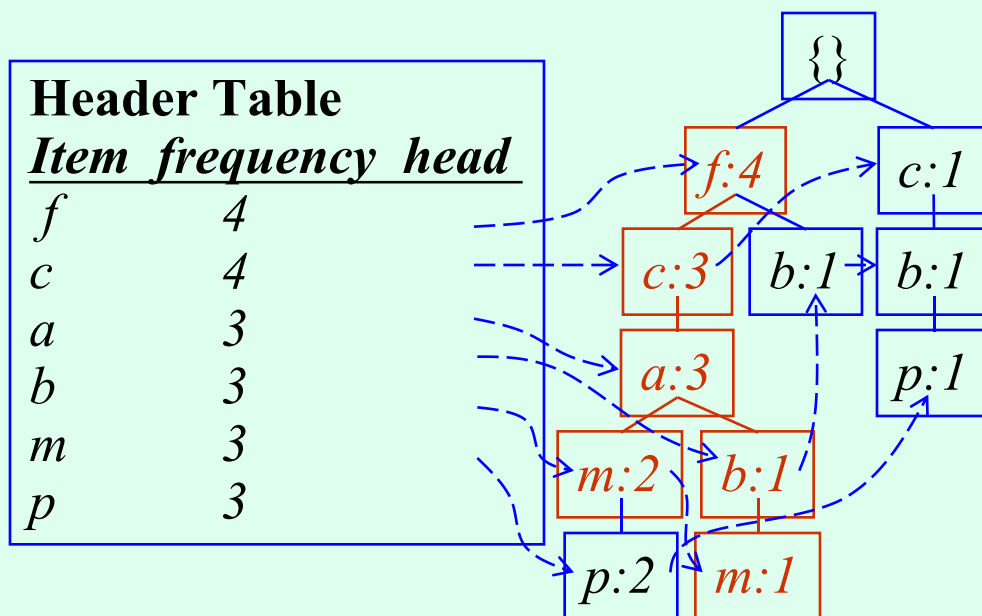
mac-conditional pattern base:
f:3



mac-conditional FP-tree

递归构造条件模式基

- 用每一个项目的条件模式基作为输入，递归FP-Growth算法



All frequent patterns
relate to *m*

m,
fm, *cm*, *am*,
fcm, *fam*, *cam*,
fcam

√ 构造FP-tree

- u 为了快速生成各个频繁集合的投影数据库，避免顺序扫描

√ 挖掘FP-tree

- u 就是一个按照深度优先顺序生成投影数据库的过程。
- u 每一个需要为其生成投影数据库的集合，都是频繁集。
不频繁集在构造FP-Tree时就过滤掉了
- u 当递归结束时，只要把所有的频繁集合并就是最后的所有频繁集

Benefits of the FP-tree Structure

✓ Completeness

- u Preserve complete information for frequent pattern mining
- u Never break a long pattern of any transaction

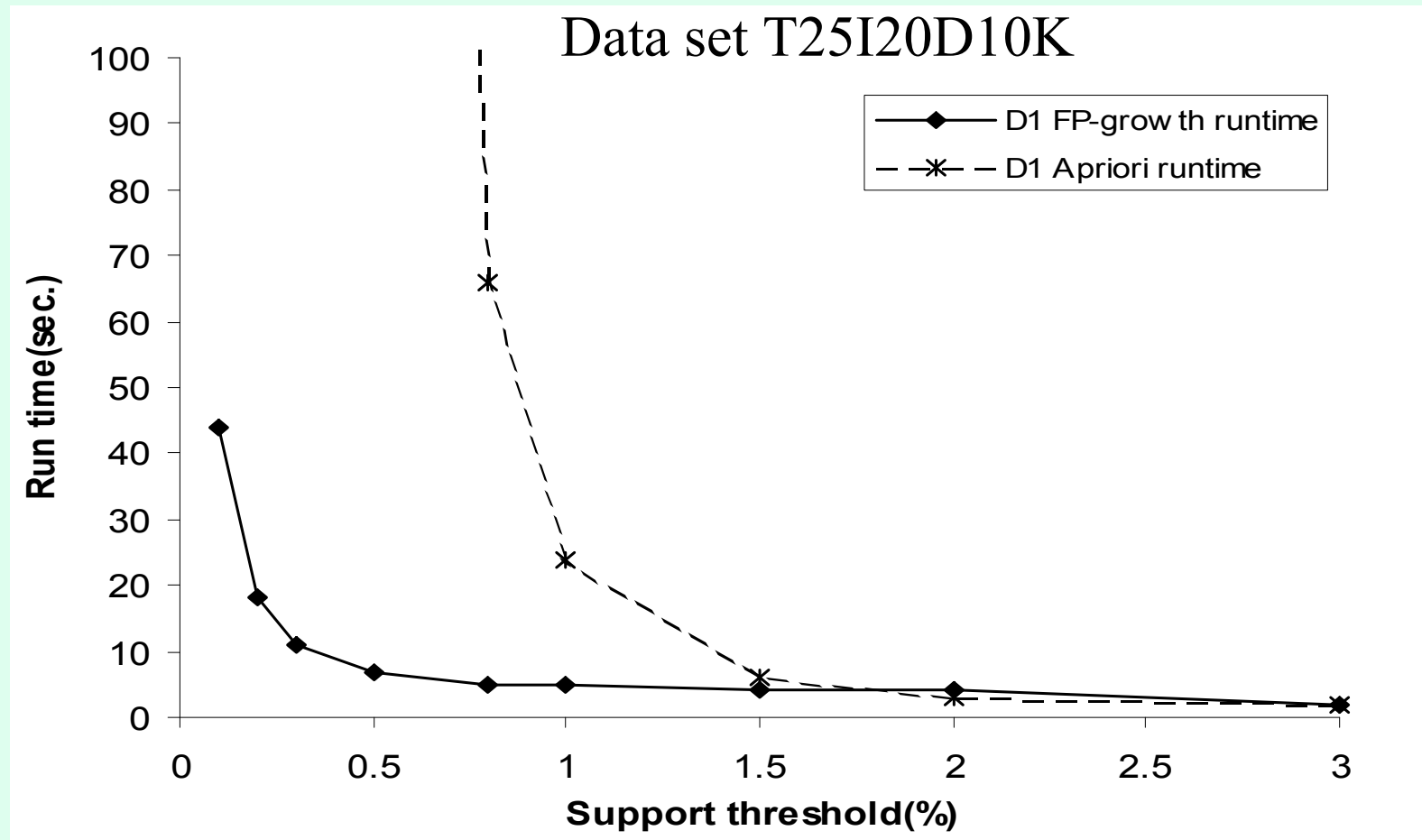
✓ Compactness

- u Reduce irrelevant info—infrequent items are gone
- u Items in frequency descending order: the more frequently occurring, the more likely to be shared
- u Never be larger than the original database (not count node-links and the *count* field)
- u For Connect-4 DB, compression ratio could be over 100

Partition Patterns and Databases

- ✓ Frequent patterns can be partitioned into subsets according to f-list
 - u F-list=f-c-a-b-m-p
 - u Patterns containing p
 - u Patterns having m but no p
 - u ...
 - u Patterns having c but no a nor b, m, p
 - u Pattern f
- ✓ Completeness and non-redundancy

FP-Growth vs. Apriori: Scalability With the Support Threshold



VIPER vs. FP-Growth: Comparision

- Compression Strategy
 - VIPER compress the bitmap for each item/itemset individually
 - In FP-tree, items at the bottom of the tree might not be compressed. Have to pay memory cost for storing pointers.
- Experiments Conduct
 - VIPER: conducted experiments on dataset up to 1.2GB
 - FP-tree: largest dataset tested on is approximately 5MB
- Locality on Secondary Storage
 - VIPER: each compressed bitmap can be stored consecutively on the disk
 - FP-tree: Don't see an obvious way since each transaction must be grouped together and they will be visited solely to retrieve one item at different point in time

VIPER vs. FP-Growth: Comparision

- But are they really different?
 - Can they be converted to the other format easily?
 - Hint: What is the complexity of merging k sorted list?
- Analogy:
 - Normalize vs Un-normalized database
 - One pay a cost to join up tables, the other save the cost of a relational table join but pay in term of storage and when only one of the attributes is needed
 - Row store vs Column Store
 - Scalable Semantic Web Data Management Using Vertical Partitioning (Best Paper Award) *Proceedings of VLDB*, 2007.
Daniel Abadi, Adam Marcus, Samuel Madden, and Katherine Hollenbach

Why would write-based algorithms be faster ?

- Read-based FPM algorithms like Apriori algorithm which will "forget" its previous processing. Eg. let the frequent itemset {a,b,c,d,e} be frequent. Let's look at a record containing {a,b,c}.

Level	Candidate	Record	
1-itemset	{a}	{TID=10: a, b , c }	← scanned
2-itemset	{a,b}	{TID=10: a, b, c }	← scanned
3-itemset	{a,b,c}	{TID=10: a, b, c}	← scanned

- Write-based FPM avoid this repeated scanning by doing depth first search

Level	Candidate	Record
1-itemset	{a}	{TID=10: a, b, c}
2-itemset	{a,b}	{TID=10: b, c}
3-itemset	{a,b,c}	{TID=10: c}

Why would write-based algorithms be faster ?

- Using storage to reduce run time complexity
- Let us see the counting process as a join algorithm
 - A set of patterns
 - A set of tuples
 - Compare tuples to patterns to see which patterns are found in which tuples
- DFS reduce the set of patterns and tuples being compared i.e. patterns will only be compared against tuples with the same prefix

However.....

- Write-based algorithm usually pay a higher cost in term of **space complexity**.
- Projected databases are materialize for **all the red circles** on the way to getting the $\{a,b\}$ projected database.

TID	Items
30	c, e

all rows that contain "ab".

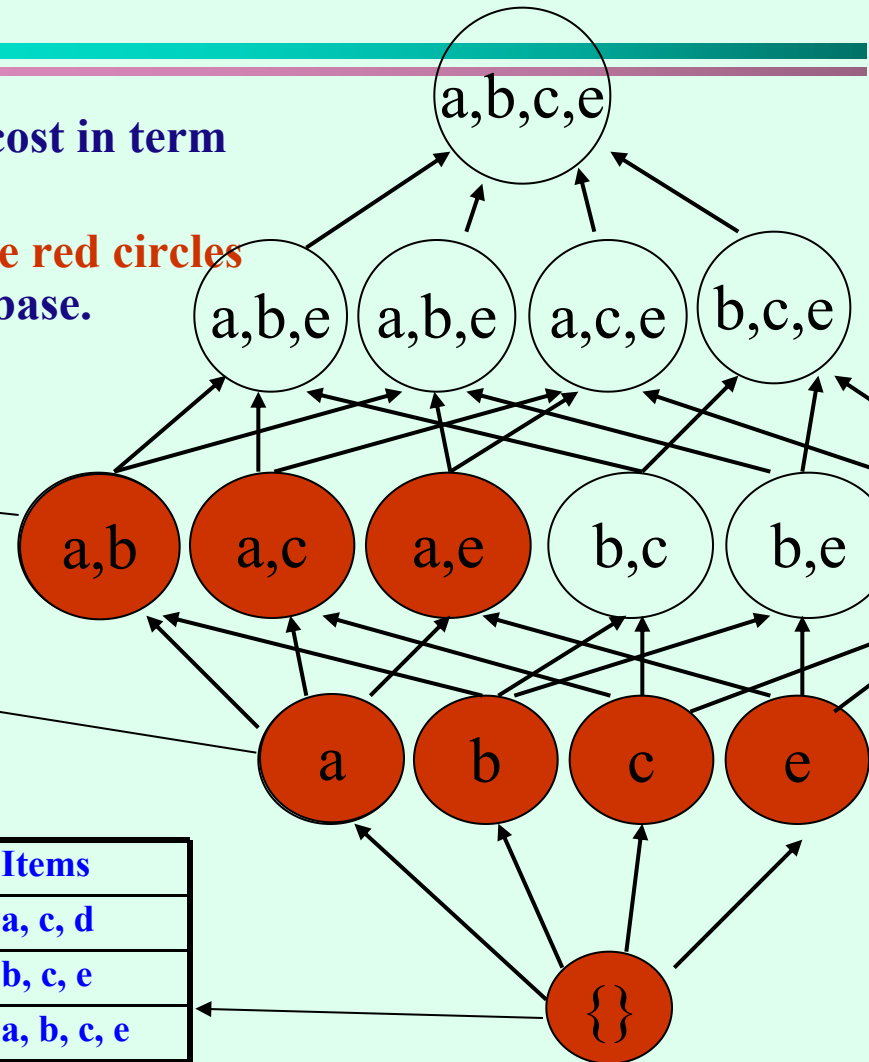
$\{ab\}$ -projected database

TID	Items
10	c, d
30	b, c, e

all rows that contain "a".

$\{a\}$ -projected database

TID	Items
10	a, c, d
20	b, c, e
30	a, b, c, e
40	b, e



TID	Items
10	a, c, d, f
20	b, c, e
30	a, b, c, e
40	b, e

a

b

c

e

TID	Items
10	c, d
30	b, c, e

TID	Items
20	c, e
30	c, e
40	e

TID	Items
10	d, f
20	e
30	e

TID	Items
20	
30	
40	

ab

ac

ae

TID	Items
30	c, e

TID	Items
10	d
30	e

TID	Items
30	e

abc

abe

No candidates ?

- There are claims that write-based algorithm are faster because they don't generate candidates. This is not true in general. Example, if minsup is 30%, then all the items in the {a}-projected database are false candidates ! (i.e. {a,b}, {a,c}, {a,d}, {a,e}, {a,f} and {a,g} are never frequent

TID	Items
10	a, c, d
20	b, c, e
30	a, b, c, e
40	a, f
50	b,c
60	a,g
70	f, g,
80	e, g
90	a,b ,g
100	b, e

{a}-projection

TID	Items
10	c, d
30	b, c, e
40	f
60	g
90	b, g

So far, we covered

	Read-based	Write-based	Point-based
Association Mining	Apriori[AgSr94]	VIPER[SHSB00] FPGrowth [HaPe00]	
Sub-Graph Mining	AGM[PKDD'00] FSG[ICDM'01]	Mofa[ICDM'02] gSpan[ICDM'02]	
Sequential Pattern Discovery	GSP[AgSr96]	PrefixSpan [PHPC01]	
Iceberg Cube	Apriori[AgSr94]		BUC[BeRa99], H-cubing [HPDW01]

本讲内容

√ 关联规则

u Write-based、DFS算法

– VIPER

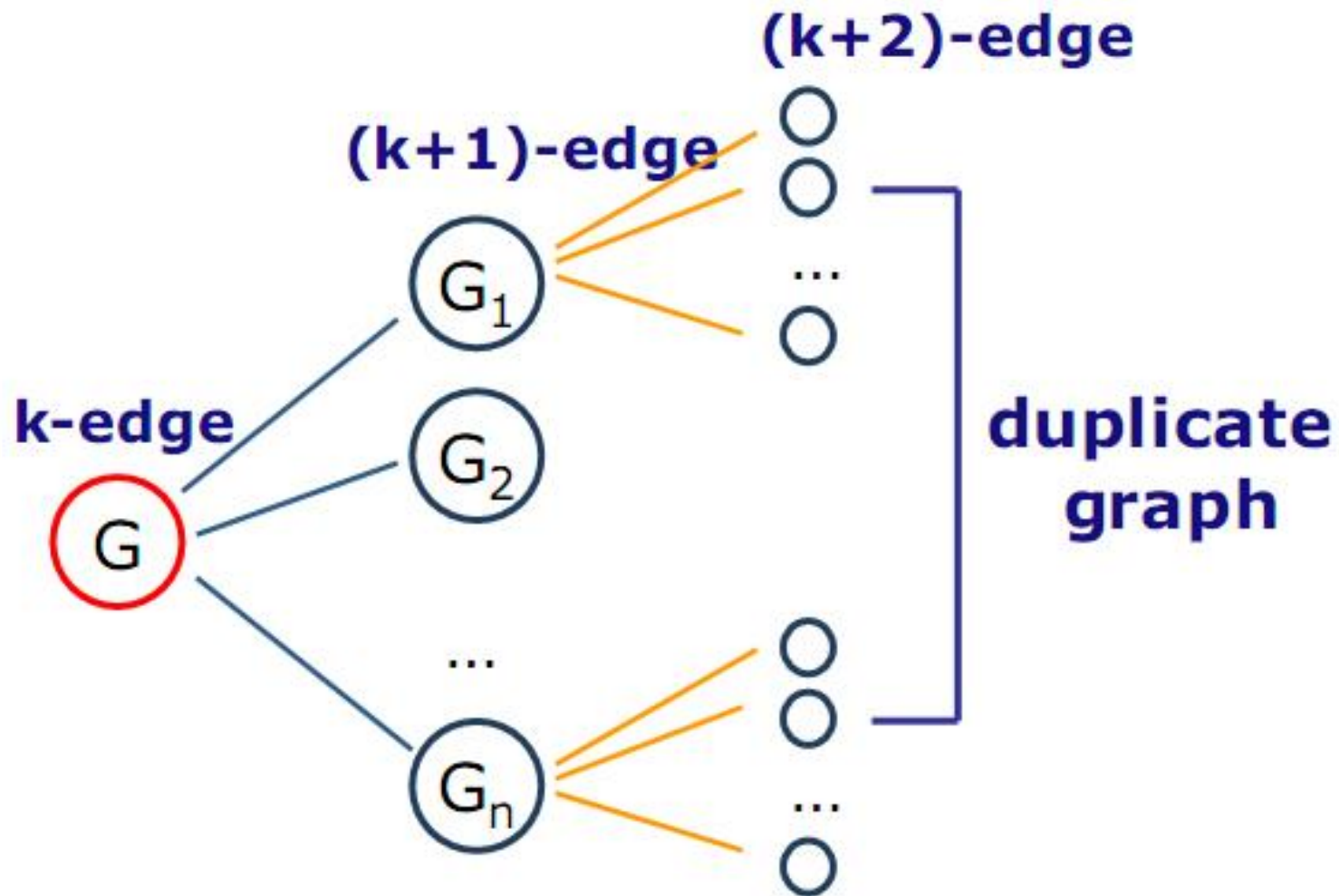
- | Implement DFS with bitmap compression of data in vertical format

– FP-Growth算法

- | Implement DFS using a tree structure in horizontal format

– FP-Growth based sub-graph mining

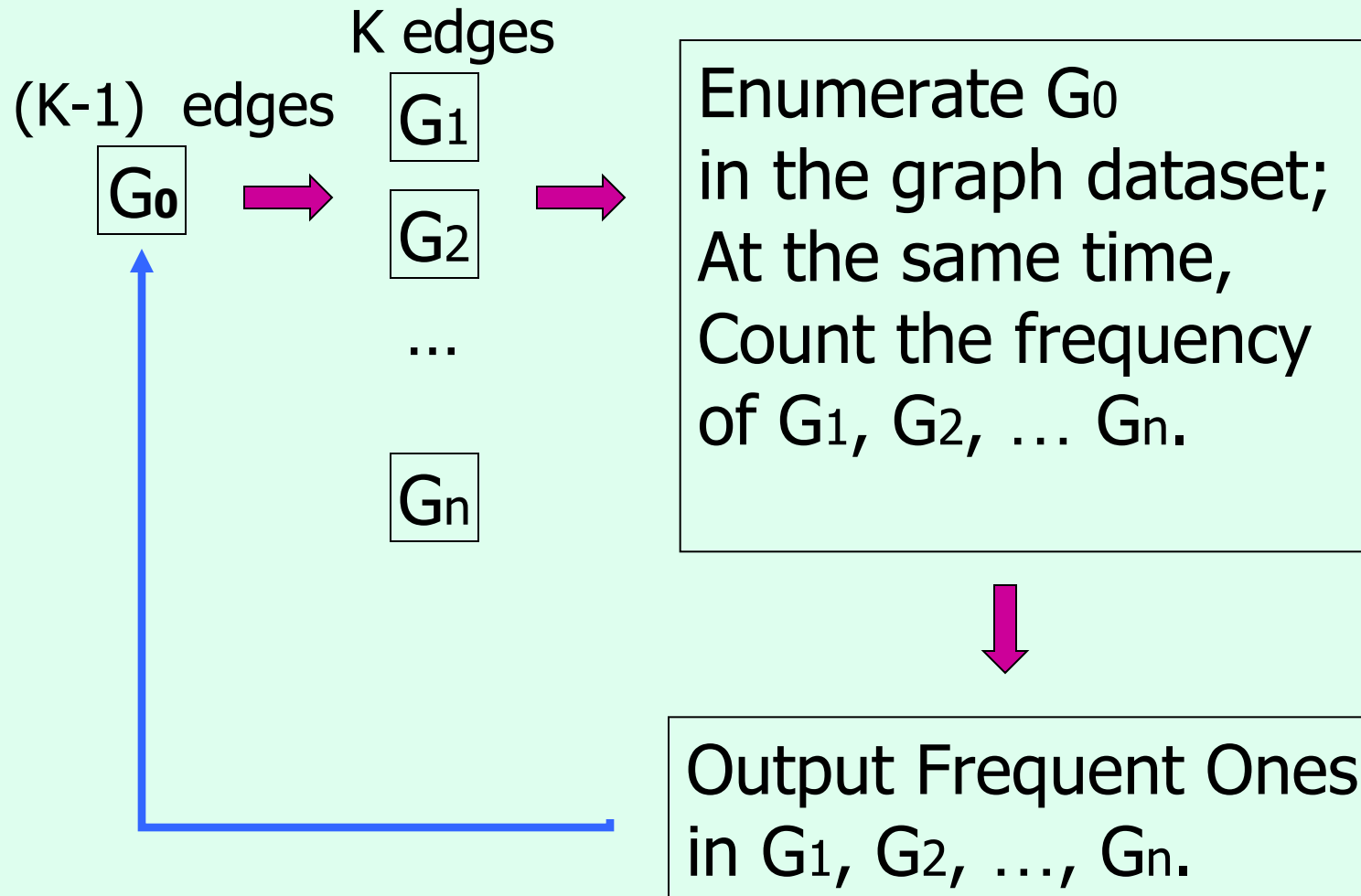
Sub-Graph Mining---pattern growth method



Mofa (ICDM'02)

- Extend graphs by adding a new edge
- Store embeddings of discovered frequent graphs
 - Fast support calculation
 - Also used in other later developed algorithms such as FFSM and GASTON
 - Expensive Memory usage
- Local structural pruning

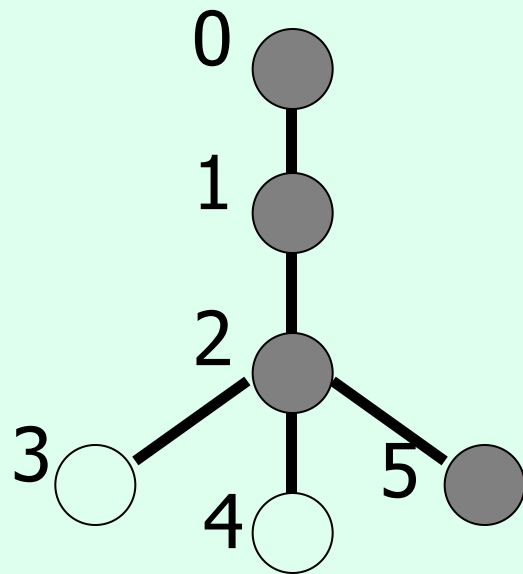
gSpan (Yan & Han, ICDM'02)



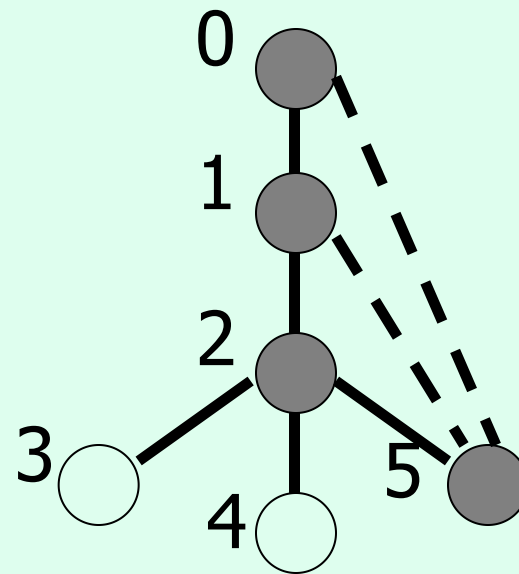
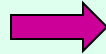
gSpan (cont.)

- ✓ **gSpan extends a k -edge graph to a $(k+1)$ -edge graph by adding one edge**
- ✓ **gSpan chooses only one depth-first search tree in a graph which produces the minimum DFS code, the canonical label of the graph**
- ✓ **gSpan only adds new edges on the vertices along the right-most path of the depth-first search tree**

Right-Most Extension



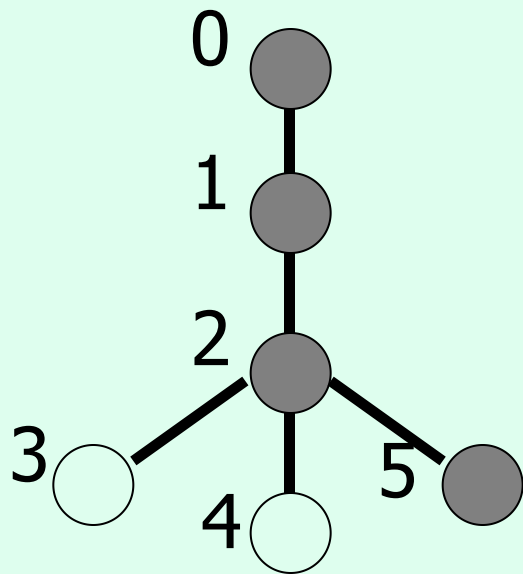
(a)



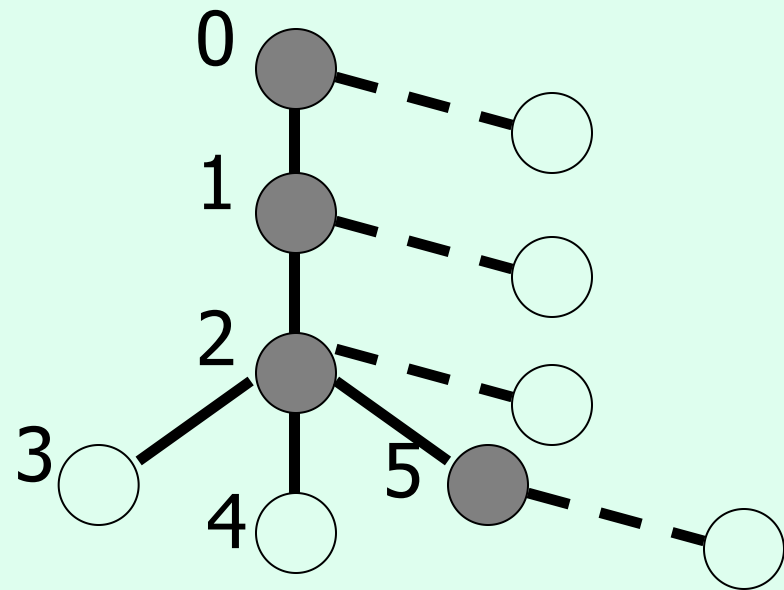
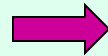
(b)

Backward-edge extension

Right-Most Extension (cont.)



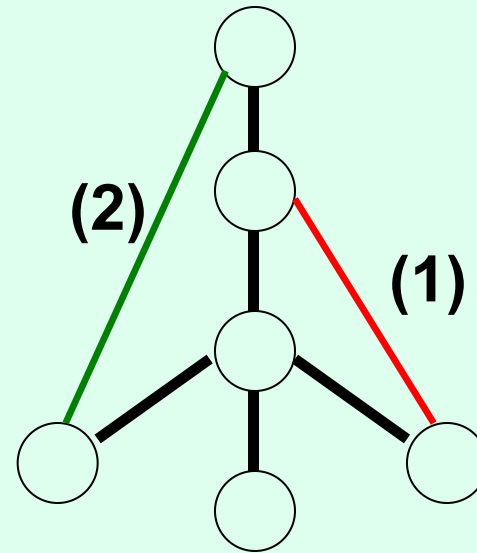
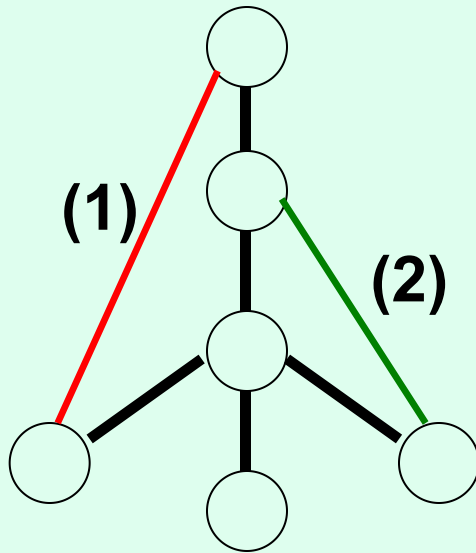
(a)



(b)

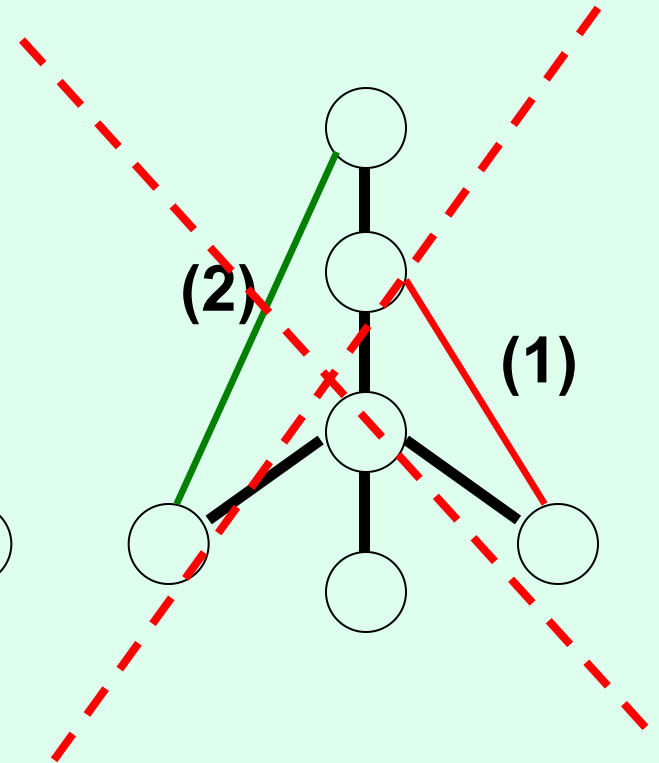
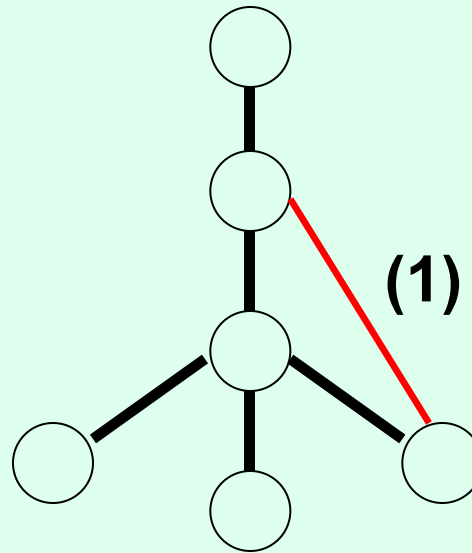
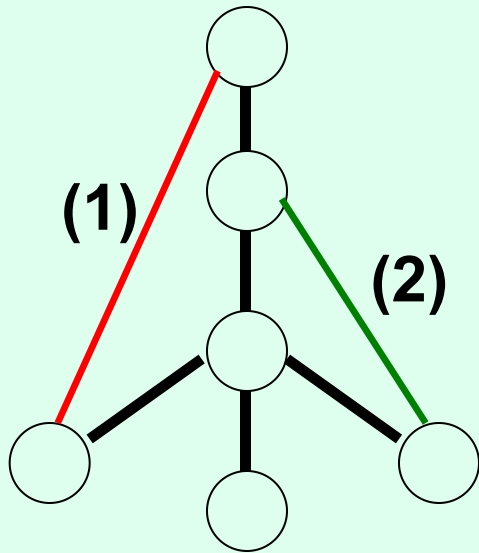
Forward-edge extension

Avoid Duplicate Extension



They are the same graph, but they grow in different ways. The cost to detect them is huge if there are lots of such duplicate growths.

Right-most Extension

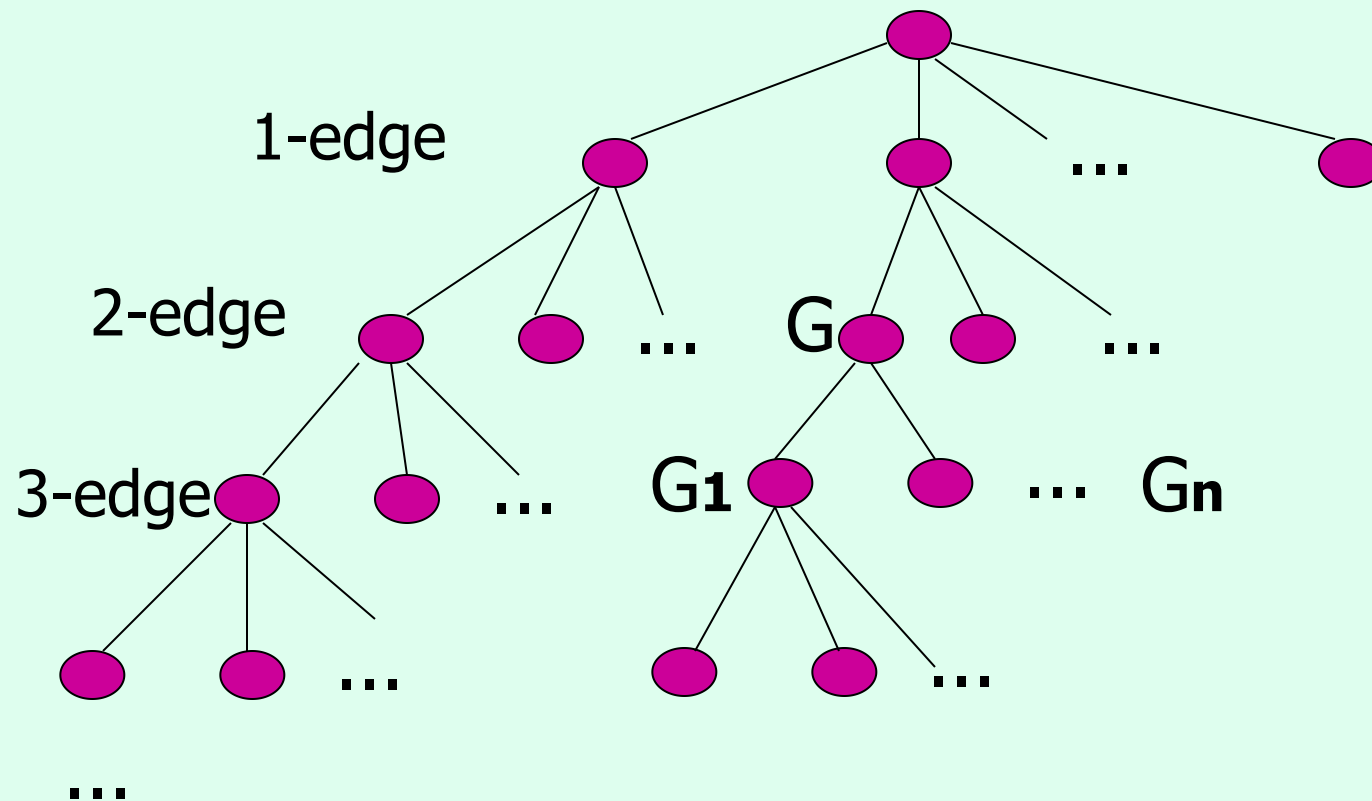


■ Theorem:

Any graph can be grown
by right-most extension

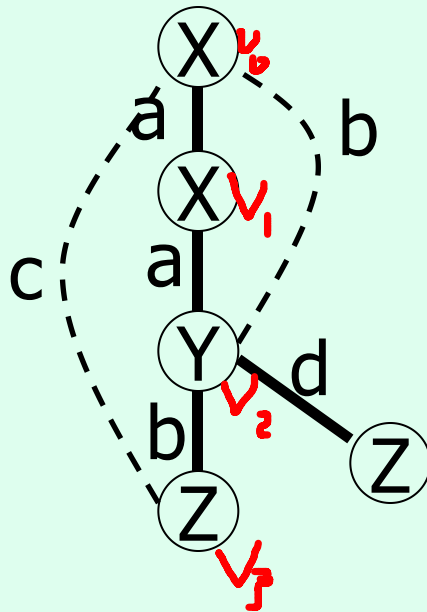
We will never generate this

Search Space in gSpan



DFS Code

Flatten a graph into a sequence such that any right-most extension takes place in the end of the sequence.



e0: (0,1,X,a,X)

e1: (1,2,X,a,Y)

e2: (2,0,Y,b,X)

e3: (2,3,Y,b,Z)

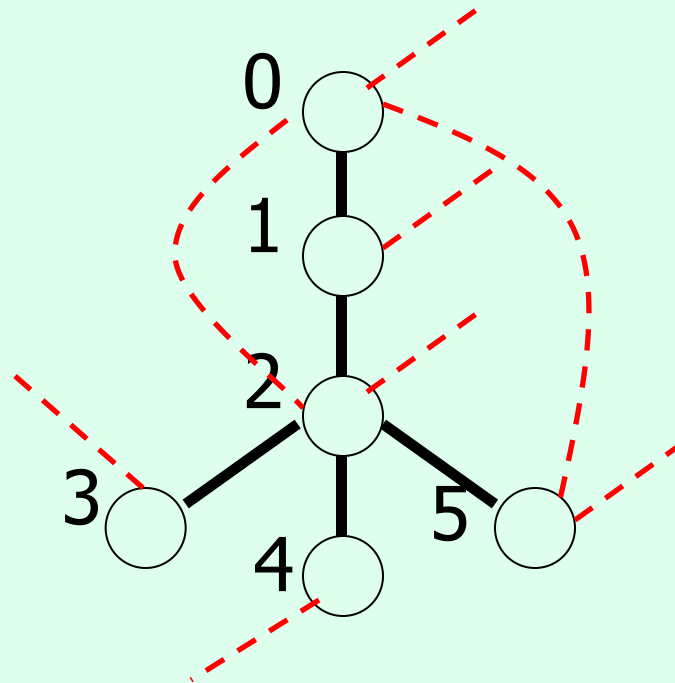
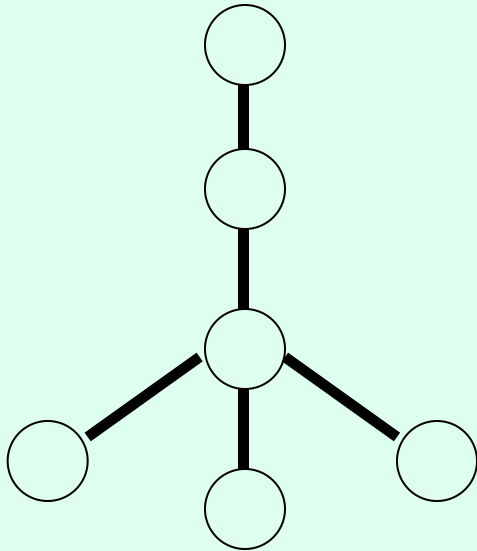
e4: (3,1,Z,c,X)

e5: (2,4,Y,d,Z)

DFS Code (cont.)

- ✓ We call a graph with a DFS tree subscripted by subscripted graph
- ✓ DFS Code $\leftrightarrow$ subscripted graph
- ✓ Actually, the construction of DFS Code is a right-most extension procedure to extend a graph
- ✓ We call a subscripting which can generate the minimum DFS code by a base subscripting
- ✓ We only conduct right-most extension on this base subscripting
- ✓ Base subscripting + one edge right-most extension MAY NOT be another base subscripting

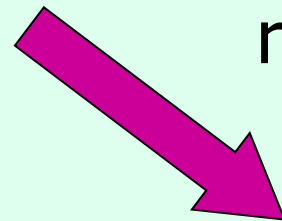
Reduce the Number of $G_1, \dots, G_n$



$$\# = 6 + 10 = 16$$

Right-Most Extension (cont.)

Number of Extensions = 16



right-most extension

Number of Extensions = 6

So far, we covered

	Read-based	Write-based	Point-based
Association Mining	Apriori[AgSr94]	VIPER[SHSB00] EPGrowth [HaPe00]	
Sub-Graph Mining	AGM[PKDD'00] FSG[ICDM'01]	Mofa[ICDM'02] gSpan[ICDM'02]	
Sequential Pattern Discovery	GSP[AgSr96]	PrefixSpan [PHPC01]	
Iceberg Cube	Apriori[AgSr94]		BUC[BeRa99], H-cubing [HPDW01]

PrefixSpan: Example of Write-based Method

SDB

SID	sequence
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>

Length-1 sequential patterns
<a>, , <c>, <d>, <e>, <f>

Having prefix <a>

Having prefix

Having prefix <c>, ..., <f>

<a>-projected database
 <(abc)(ac)d(cf)>
 <(_d)c(bc)(ae)>
 <(_b)(df)cb>
 <(_f)cbc>

-projected database

Length-2 sequential patterns
 <aa>, <ab>, <(ab)>,
 <ac>, <ad>, <af>

Having prefix <aa>

Having prefix <af>

<aa>-proj. db

...

<af>-proj. db

寻找频繁序列 – 朴素算法

- √ 对于每条序列数据，找出它所有的子序列（找出所有可能的子序列T）
- √ T在整个数据集中做匹配，计算它作为子序列的频率
- √ 若频率大于阈值 α ，则T是一个频繁序列

- √ 记序列数据平均长度为l，共有n条序列数据
- √ 时间复杂度 $O(n2^l * nl)$
- √ **Too SLOW~~~~~**

寻找频繁序列 – 优化

- √ 慢在哪里?
- √ 枚举太多非频繁的子序列

u <cba>

u <eg>

u <gf>

u

u 甚至<g>本身就不是频繁序列

SID	sequence
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>

寻找频繁序列 – 优化

- √ 观察<g>和<gf>
- √ 他们都不是频繁序列
- √ <g>是<gf>的前缀
- √ 前缀都不是频繁序列
- √ 整体可能是频繁序列吗?

SID	sequence
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>

- √ 利用前缀，在枚举过程中，去除不必要的子序列（剪枝）

PrefixSpan算法

- Prefix-Projected Pattern Growth, 前缀投影的模式挖掘。
- S的投影: 在某序列数据中, 以S为前缀的最长子序列的后缀
- 以序列数据<a(abc)(ac)d(cf)>为例

前缀S	S的投影
<a>	<(abc)(ac)d(cf)>
<aa>	<(_bc)(ac)d(cf)>
<ab>	<(_c)(ac)d(cf)>
<cd>	<(cf)>

PrefixSpan算法 – 流程

- √ 先找长度为1的子序列（单个字母）
- √ 筛出其中的频繁序列，称他们为S1
- √ 计算每个S1在每条序列数据中的投影
- √ 对于每个S1，在其投影中找一个字母连上它

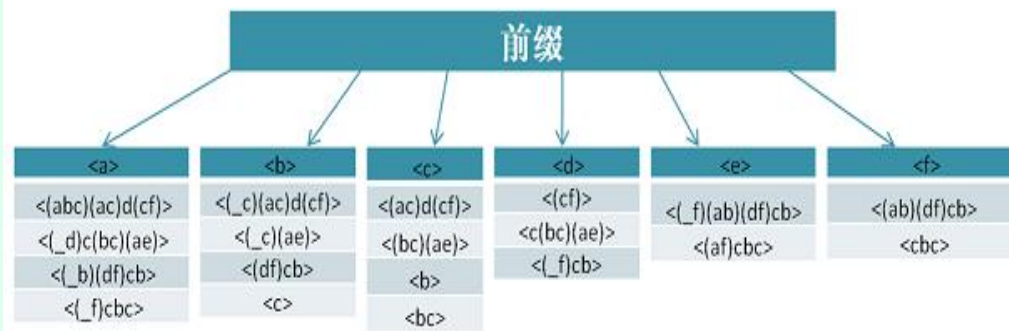
- √ 得到长度为2的子序列（2个字母）
- √ 筛出其中的频繁序列，称他们为S2
- √ 计算每个S2在每条序列数据中的投影
- √ 对于每个S2，在其投影中找一个字母连上它

- √
- √ 直到某次已经没有频繁序列，算法结束

id	Sequence
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>

<a><c><d><e><f>

<a>		<c>	<d>	<e>	<f>	<g>
4	4	4	3	3	3	1



PrefixSpan算法 – 流程

√ 以<d>为例运行算法

① <d>

② <d>: <dc> <df> <db> <da> <de> <(df)>

③ <dc>: <d(cf)> <dcb> <dca> <dce>

③ <db>: <d(bc)> <dba> <dbe>

④ <dcb>: <dc(bc)> <dcba> <dcbe>

找不出更长的频繁序列，算法的<d>分支结束。

id	Sequence
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>

<a><c><d><e><f>

<a>		<c>	<d>	<e>	<f>	<g>
4	4	4	3	3	3	1

<dcb>
<(_c)(ae)>
<(bc)(ae)>

PrefixSpan算法 – 流程

- √ 用类似的方法可以找出其他以
- √ $\langle a \rangle \langle b \rangle \langle c \rangle \langle e \rangle \langle f \rangle$ 为前缀的频繁序列

PrefixSpan算法 – 小结

- √ 基于一个直观思想:
- √ 若S都不是频繁序列，则以S为前缀的子序列更不可能是频繁序列
- √ 与朴素算法比较:

	PrefixSpan	朴素
枚举子序列	长度增加，子序列数量急剧增长 更难成为频繁序列，大量子序列被筛去 开销保持稳定较小	长度增加，子序列数量急剧增长 没有筛去无用子序列（剪枝） 开销极大
检验频繁序列	只在每条序列数据的投影里匹配，开销不断缩小	在每条完整的序列数据里匹配，开销大
总结	时空开销小，算法结束快	时空开销大，运行时间长

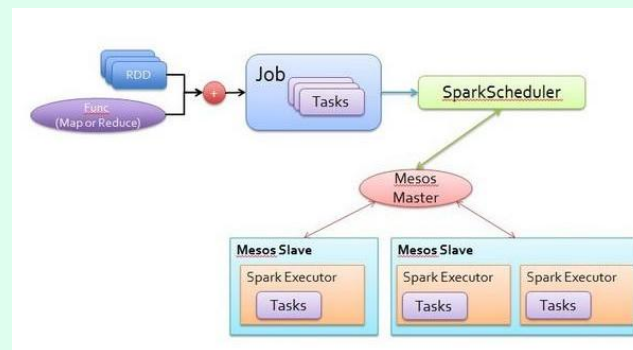
- √ 与其他算法（GSP、FreeSpan）比较:
- √ 优势较大，主要在于投影数据集收缩较快

PrefixSpan算法 – 小结

- √ PrefixSpan算法的时间复杂度并没有减小
- √ 只是因为筛去无用子序列从而在实际应用中表现较好
- √ 极端情形:
 - √ 每条序列数据都只有微小差别
- √ 现实中很难出现
 - √ 高度相似的采样可以被合并处理

PrefixSpan算法 – 拓展

- ✓ PrefixSpan具有很好的可并行性
- ✓ 例如：以<a>和<d>为前缀的子序列是不同（独立）的
- ✓ 因此，可以并行搜索以其为前缀的频繁序列
- ✓ 在分布式大数据平台中能够很好地提升性能
- ✓ 例如：Spart的Mllib



So far, we covered

	Read-based	Write-based	Point-based
Association Mining	Apriori[AgSr94]	VIPER[SHSB00] EPGrowth [HaPe00]	
Sub-Graph Mining	AGM[PKDD'00] FSG[ICDM'01]	Mofa[ICDM'02] gSpan[ICDM'02]	
Sequential Pattern Discovery	GSP[AgSr96]	PrefixSpan [PHPC01]	
Iceberg Cube	Apriori[AgSr94]		BUC[BeRa99], H-cubing [HPDW01]

References

- v [AgSr94] R. Agrawal, R. Srikant, "Fast Algorithms for Mining Association Rules", Proc. of the 20th Int'l Conference on Very Large Databases, Santiago, Chile, Sept. 1994.
- v [AgSr96] R. Srikant, R. Agrawal: "Mining Sequential Patterns: Generalizations and Performance Improvements", to appear in *Proc. of the Fifth Int'l Conference on Extending Database Technology (EDBT)*, Avignon, France, March 1996.
- v [BeRa99] Kevin S. Beyer and Raghu Ramakrishnan. "Bottom-Up Computation of Sparse and Iceberg CUBEs". In Proceedings of the ACM SIGMOD International Conference on Management of Data, pages 359-370, June 1999.
- v [HPDW01] J. Han, J. Pei, G. Dong, and K. Wang, "Efficient Computation of Iceberg Cubes with Complex Measures", Proc. 2001 ACM-SIGMOD Int. Conf. on Management of Data (SIGMOD'01), Santa Barbara, CA, May 2001.
- v [HPY00] J. Han, J. Pei, and Y. Yin, ``Mining Frequent Patterns without Candidate Generation`, Proc. 2000 ACM-SIGMOD Int. Conf. on Management of Data (SIGMOD'00), Dallas, TX, May 2000.
- v [HaPe00] J. Han and J. Pei ``Mining Frequent Patterns by Pattern-Growth: Methodology and Implications`, ACM SIGKDD Explorations (Special Issue on Scalable Data Mining Algorithms), 2(2), December 2000.
- v [PHPC01] J. Pei, J. Han, H. Pinto, Q. Chen, U. Dayal, and M.-C. Hsu, "PrefixSpan: Mining Sequential Patterns Efficiently by Prefix-Projected Pattern Growth", Proc. 2001 Int. Conf. on Data Engineering (ICDE'01), Heidelberg, Germany, April 2001.

References

- ✓ [SHSB00] P. Shenoy, J. Haritsa, S. Sudarshan, G. Bhalotia, M. Bawa, D. "[Turbo-Charging Vertical Mining of Large Database](#)." Proc. 2000 ACM-SIGMOD Int. Conf. on Management of Data (SIGMOD'00), Dallas, TX, May 2000.
- ✓ [Zaki98] Mohammed J. Zaki, "[Efficient Enumeration of Frequent Sequences](#)", 7th International Conference on Information and Knowledge Management, pp 68-75, Washington DC, November 1998.
- ✓ [Zaki00] Mohammed J. Zaki, "[Scalable Algorithms for Association Mining](#)", in IEEE Transactions on Knowledge and Data Engineering, Vol. 12, No. 3, pp 372-390, May/June 2000.
- ✓ [Zaki01] Mohammed J. Zaki, "[SPADE: An Efficient Algorithm for Mining Frequent Sequences](#)", in Machine Learning Journal, special issue on Unsupervised Learning (Doug Fisher, ed.), pp 31-60, Vol. 42 Nos. 1/2, Jan/Feb 2001.